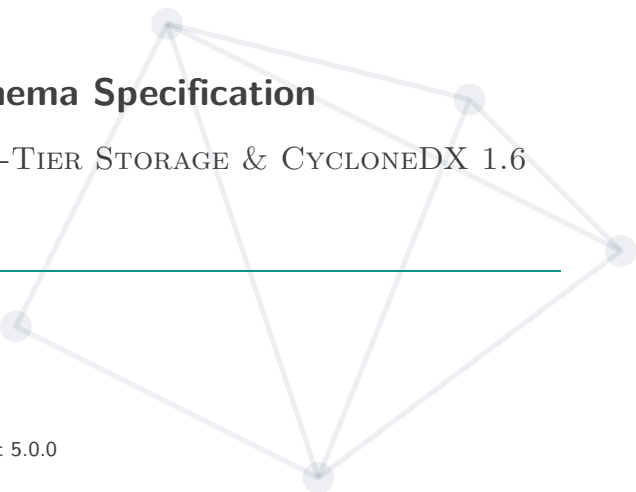


AI-BOM Architecture & Schema Specification



EVIDENCE-BASED IDENTITY, TWO-TIER STORAGE & CYCLONEDX 1.6
ML-BOM

AI-BOM - SCA

Ngày 2 tháng 9 năm 2026

AI Software Bill of Materials Engine 0.1.0 · Product 5.0.0

Tổng Quan Vấn Đề & Nguyên Tắc Thiết Kế

Crawl Dữ Liệu Hugging Face & Cache SQLite

Quy Tắc Đánh Giá Độ Tin Cậy & Xử Lý Xung Đột

12 Tình Huống Thực Tế Khi Quét Model & JSON Mẫu

Cấu Trúc JSON Đầu Ra: Chuẩn Quốc Tế & Nội Bộ

Đọc Header Binary, Chống Mã Độc & Kết Quả Test

Tổng Quan Vấn Đề & Nguyên Tắc Thiết Kế

Tóm Tắt Giải Pháp (Executive Summary)

! Thực Trạng

- Không có centralized registry cố định → mô hình được phân phối mở.
- Model weights bị re-upload diện rộng → metadata phân mảnh và bất đồng.
- Model Card khai báo thủ công → thiếu cơ chế xác minh tự động.

🛡 Đề Xuất

- **4 cấp Evidence Class:** Đo đạc cấu trúc binary trực tiếp > Declared metadata.
- **SQLite 2 tầng:** Offline replayability, quét hoàn toàn độc lập với mạng.
- **CycloneDX 1.6:** Chuẩn hóa quốc tế, minh bạch evidence.

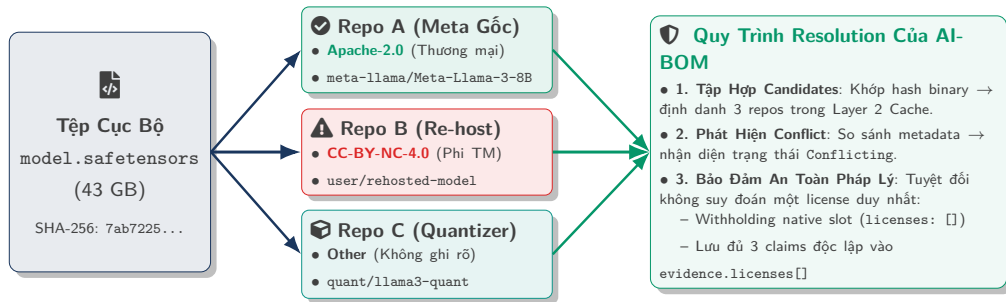
✔ Dữ Liệu Thực Nghiệm

Đã kiểm thử trên **49,992 models** (798,682 file hashes, DB nén 912 MB).

So Sánh: SBOM Phần Mềm Truyền Thống vs AI-BOM

Tiêu chí	Software SBOM (npm/Cargo)	AI-BOM (Hugging Face/GGUF)
1. Phân phối	Registry tập trung, version bất biến	Hub phân phối mở, weights được tái chia sẻ diện rộng
2. Bản chất File	Source code / text đọc được	Matrix weights nhị phân lớn (GB–TB)
3. Quan hệ Hash	1 Version ↔ 1 SHA-256 duy nhất	1 Hash → 1,037 Repos (Trùng lặp lớn)
4. Metadata	Khai báo chuẩn qua lockfile	YAML Model Card khai báo thủ công
5. License	SPDX rõ ràng, chuẩn 1–1	Conflict license giữa repo gốc và các bản re-upload

Sơ Đồ Trực Quan: Cơ Chế Nhận Diện & Xử Lý Xung Đột Đa Repo (1-N)



💡 Nguyên Tắc An Toàn Pháp Lý

Khi phát hiện conflict bản quyền giữa các candidates của cùng một model weights, hệ thống áp dụng withholding cho native slot và lưu giữ đầy đủ claims trong evidence phục vụ kiểm toán độc lập.

5 Nguyên Tắc Thiết Kế Cốt Lõi



1. Định Danh Theo File Thực Tế

Tính danh tính từ hash binary thật, không dựa vào đường dẫn tệp.



2. Minh Bạch Provenance

Ghi nhận xuất xứ dữ liệu (provenance) tường minh qua từng nguồn (`local_binary`, `hf_card`).



3. Nguyên Tắc Withholding & Zero-Guesswork

Khi thiếu bằng chứng xác thực hoặc phát hiện conflict → áp dụng withholding (`withheld`), không suy đoán dữ liệu.



4. Tách Bằng Chứng & Kết Luận

Lưu cả dữ liệu thô (`claims[]`) và kết luận (`concludedValue`).



5. Kết Quả Tất Định

Cùng input → output JSON trùng byte-for-byte (RFC 9562 UUIDv8).

6 Nguyên Tắc Bất Biến Trong Thiết Kế Hệ Thống

#	Nguyên Tắc Bất Biến	Mục Đích & Hành Vi Kỹ Thuật
1	Offline Replayability	Rebuild toàn bộ Cache L2 từ Raw L1 với 0 request mạng .
2	Minh Bạch Bằng Chứng	Output JSON luôn chứa cả <code>claims[]</code> thô và kết luận resolution.
3	Zero-Guesswork	Dữ liệu conflict → duy trì trạng thái tranh chấp, không suy đoán chủ quan.
4	Zero-Network Core	Crate <code>aibom-core</code> độc lập hoàn toàn với network stack.
5	Loại Bỏ File Rỗng	Bỏ qua hash file 0-byte khi tính danh tính weight set của model.
6	Đối Chiếu An Toàn	Chỉ nạp config ngoài khi thư mục chỉ chứa đúng 1 model.

Crawl Dữ Liệu Hugging Face & Cache SQLite

Số Liệu Thực Tế Khi Crawl 50.000 Models (data/poc-50k)

Chỉ số đo đạc	Dataset 20k	Dataset 50k (Hiện tại)
Tổng số Models	19,998	49,992 / 50,000
Tổng số File Hashes	268,052	798,682 hashes
File Weights	267,979	705,158 (93.85%)
Distinct Weight Sets	19,235	47,854 sets
Hash trùng đa Repo	6.10%	5.11% (max 1,037)
Repo đa nhóm Weight	57.4%	65.1% repos
Dung lượng DB L2	≈360 MB	≈ 912 MB (18 KB/model)



Nhận Xét Thực Tế

- 5.11% hash xuất hiện ở > 1 repo
→ hiện tượng trùng file là bản chất của AI Hub.
- 65.1% repo chứa > 1 nhóm weights → bắt buộc phải tách nhỏ theo từng nhóm.

Những "Bẫy Dữ Liệu" Trên Hugging Face Hub

📄 1. README Không Chuẩn Hóa

Tác giả nhúng toàn văn bài báo vào YAML → metadata phình to hàng chục MB văn bản phi cấu trúc.

🔗 2. File Dùng Chung Trùng Hàng Nghìn Repo

File `tokenizer.json` xuất hiện ở **1,037 repos** với 12 license khác nhau.

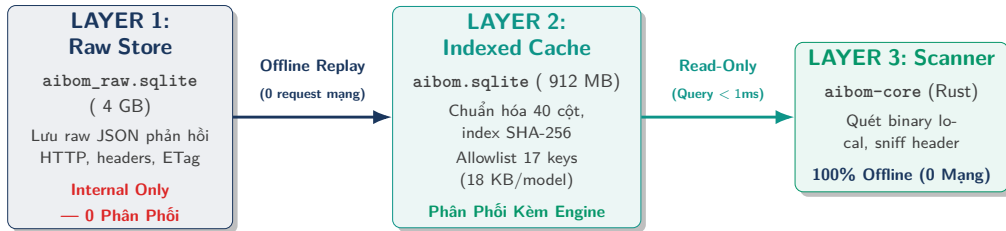
↔ 3. Conflict Trong Cùng 1 Repo

Cùng 1 model: Tags ngoài ghi Apache-2.0 nhưng Card trong ghi MIT.

📦 4. 1 Repo Chứa Nhiều Model Con

1 repo chứa 7 file GGUF quantization khác nhau (Q2, Q4, FP16).

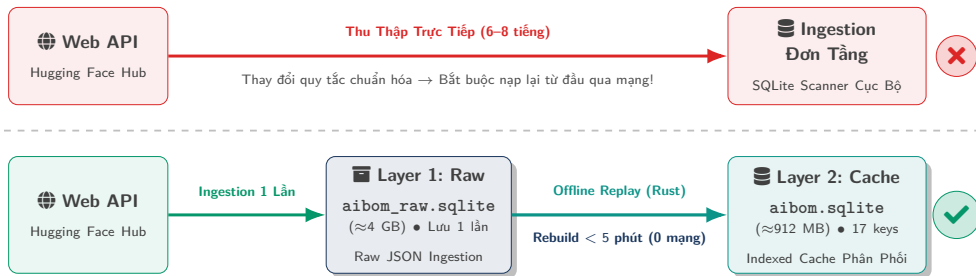
Kiến Trúc SQLite 2 Tầng: Tách Biệt Raw Data & Cache



💡 Lợi Ích Của Thiết Kế 2 Tầng

- **Gọn nhẹ:** DB phân phối kèm scanner chỉ nặng 912 MB (18 KB/model).
- **Offline Replayability:** Khi cập nhật quy tắc chuẩn hóa, rebuild L2 từ L1 trong vài phút, 0 cần thu thập lại qua mạng.

Sơ Đồ Trực Quan: Cơ Chế Offline Replayability (Layer 1 → Layer 2)



Ý Nghĩa Kiến Trúc

Tách biệt hoàn toàn tầng lưu trữ nguyên thủy (Raw Ingestion) và tầng lập chỉ mục (Clean Projection). Giúp kỹ sư thử nghiệm hàng chục bộ quy tắc chuẩn hóa mà không gây nghẽn mạng hay bị khóa API.

Bộ Lọc 17 Trường Quan Trọng Từ Model Card (Allowlist)

☰ 17 Trường Cần Thiết Được Giữ Lại

Chỉ giữ đúng 17 trường có giá trị kiểm toán:

- | | |
|-----------------|----------------------|
| 1. license | 10. model_name |
| 2. library_name | 11. license_name |
| 3. pipeline_tag | 12. license_link |
| 4. tags | 13. widget |
| 5. datasets | 14. co2_eq_emissions |
| 6. metrics | 15. fine_tuned_from |
| 7. base_model | 16. thumbnail |
| 8. language | 17. model_type |
| 9. inference | |

⚡ Hiệu Quả Nén Dữ Liệu

- Loại bỏ văn bản phi cấu trúc dư thừa → dung lượng nén đạt **18.0 KB / model**.
- Toàn bộ 50.000 models lưu gọn trong **912 MB SQLite**.

Bảng Model & Xử Lý Model Khai Báo License "Other"

```
CREATE TABLE model (  
  repo_id TEXT PRIMARY KEY, author TEXT, model_name TEXT, sha TEXT,  
  last_modified TEXT, private INTEGER, gated INTEGER, disabled INTEGER,  
  pipeline_tag TEXT, library_name TEXT, model_type TEXT, model_architecture TEXT,  
  license_spdx TEXT,  
  license_name TEXT, -- Lưu tên license gốc khi khai báo "other"  
  license_link TEXT, -- Lưu link điều khoản license gốc  
  downloads INTEGER, likes INTEGER, created_at TEXT,  
  config_json TEXT, card_data_json TEXT, card_text_sha256 TEXT,  
  weight_set_count INTEGER, weight_file_count INTEGER, total_weight_bytes INTEGER  
  -- Tổng cộng 40 cột chuẩn hóa  
);
```

✓ Chuẩn Hóa 70.0% Model Khai Báo License "Other"

Tách riêng 2 cột license_name & license_link → chuẩn hóa 3,466 / 4,950 models khai other (đạt 70.0%).

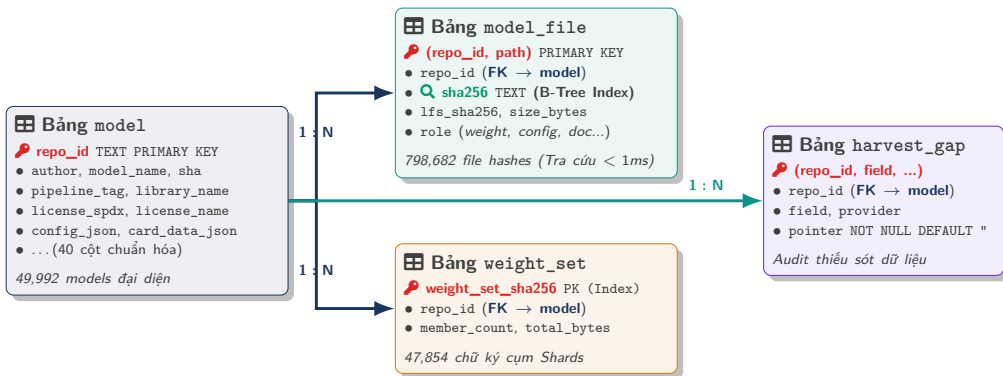
Bảng File, Weight Set & Ảnh Xạ FileRole

```
CREATE TABLE model_file (  
    repo_id TEXT, path TEXT, size_bytes INTEGER, sha256 TEXT, lfs_sha256 TEXT,  
    role TEXT, -- weight / config / tokenizer / license / doc / other (DDL)  
    PRIMARY KEY (repo_id, path)  
);  
  
CREATE TABLE weight_set (  
    weight_set_sha256 TEXT PRIMARY KEY, -- Hash tổng hợp của weight group  
    repo_id TEXT, member_count INTEGER, total_bytes INTEGER  
);
```

Ảnh Xạ 6 Role DDL Thô → 5 Cấp FileRole Trong Scanner

- DB DDL (6 role thô): weight, config, tokenizer, license, doc, other.
- Scanner Engine (5 FileRole): Weight (measured), Auxiliary (khóa trần heuristic cho config/tokenizer/license/doc), Media, Archive, Other.

Sơ Đồ Thực Thể - Quan Hệ CSDL Cache L2 (aibom.sqlite)



🔍 Tối Ưu Hóa Tốc Độ Tra Cứu (Zero-Scan Overhead)

Chỉ mục B-Tree trên cột `sha256` và `weight_set_sha256` cho phép scanner tra cứu tức thì danh tính model và candidate repos trong **dưới 1ms**, hoàn toàn độc lập với kích thước cơ sở dữ liệu.

Bảng Ghi Nhận Thiếu Sót (Gaps) & Tối Ưu DB

```
CREATE TABLE harvest_gap (  
    repo_id TEXT NOT NULL,  
    field TEXT NOT NULL,  
    provider TEXT NOT NULL,  
    pointer TEXT NOT NULL DEFAULT '', -- Tránh lỗi NULL của SQLite  
    PRIMARY KEY (repo_id, field, provider, pointer)  
);
```



Khóa Chính 4 Cột Tránh Lọt Bản Ghi Trùng

SQLite coi NULL != NULL → dùng pointer NOT NULL DEFAULT '' để bảo đảm lọc trùng chính xác 100%.

❄ Đóng Băng File SQLite Trước Khi Ship

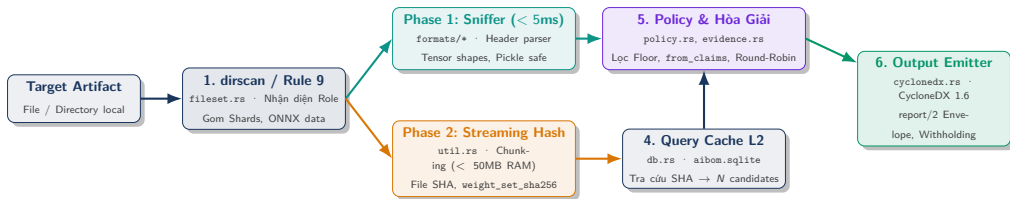
Lệnh tối ưu trước khi release:

```
PRAGMA journal_mode = DELETE;  
PRAGMA wal_autocheckpoint = 0;  
VACUUM;
```

- 1 file duy nhất, read-only, không sinh file phụ -wal.

Quy Tắc Đánh Giá Độ Tin Cậy & Xử Lý Xung Đột

Sơ Đồ Luồng Thực Thi Runtime Của aibom-core



⚙️ Quy Trình Thực Thi Khép Kín 100% Offline

Dữ liệu di chuyển tuần tự qua 7 module Rust độc lập, không chia sẻ trạng thái toàn cục, bảo đảm tính tất định và an toàn bộ nhớ.

Hệ Thống 4 Cấp Evidence Class (Evidence Levels)

heuristic (0) < declared (1) < verified (2) < measured (3)

Cấp	Evidence Class	Định Nghĩa	Phương Pháp Thu Thập
0	Heuristic	Phỏng đoán ngoại quan / regex tên tệp	Đuôi .safetensors → phỏng đoán framework
1	Declared	Declared metadata trên web (chưa xác minh)	Model Card ghi license: mit
2	Verified	≥ 2 nguồn độc lập xác thực lẫn nhau	Header nhị phân khớp với Model Card
3	Measured	Đo đạc trực tiếp từ cấu trúc binary	Binary sniffer parse tensor header

⚙️ Tại Sao Dùng Thang Thứ Tự (Ordinal Scale)?

Thang thứ tự (Ord trong Rust) giúp so sánh evidence class hoàn toàn tất định, loại bỏ hoàn toàn các điểm số xác suất mơ hồ.

Ví Dụ Thực Tế: 4 Cấp Evidence Class Khi Xác Định Param Count

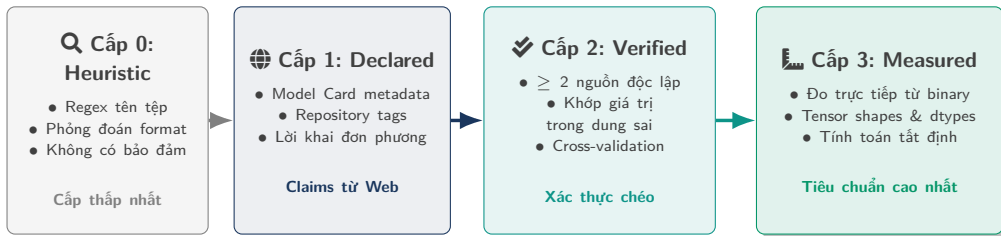
🔍 Cấp 0: Heuristic & Cấp 1: Declared

- **Cấp 0 (Heuristic):** Phân tích tên tệp llama-3-8b.gguf → Phỏng đoán sơ bộ model quy mô khoảng 8 tỷ tham số (8B).
- **Cấp 1 (Declared):** Đọc Model Card trên web → Tác giả khai báo trong README: param_count: 8B.

✅ Cấp 2: Verified & Cấp 3: Measured

- **Cấp 2 (Verified):** Web ghi 8B, binary đo ra 8.03B (khớp dung sai cho phép) → Thăng cấp **Verified**.
- **Cấp 3 (Measured):** Binary sniffer parse tensor header → Đếm chính xác **8,030,261,248 params** (Đo đạc tất định).

Sơ Đồ Phân Cấp & Hòa Giải Evidence Class



⚖️ Nguyên Tắc Hòa Giải (Resolution Policy)

- **Xác thực chéo (Cross-Validation)**: Khi giá trị khai báo (declared) khớp với cấu trúc nhị phân đo đạc (measured), thăng cấp thuộc tính lên verified.
- **Quyền ưu tiên cấu trúc (Structural Override)**: Khi có conflict giữa khai báo và đo đạc, giá trị measured luôn được chọn cho các thuộc tính vật lý, đồng thời lưu toàn bộ claims vào evidence để đối soát.

Xử Lý Conflict Giữa Declared Metadata & Measured Data

Evidence Class	Phương Pháp Thu Thập	Hành Vi Kỹ Thuật Trong Engine
1. Declared	Khai báo trong Model Card	Card ghi: param_count: 7B
2. Verified	2 nguồn độc lập khớp nhau trong sai số	Khớp dung sai ($6.74B \approx 7B$) → Verified
3. Measured	Binary sniffer parse cấu trúc tensor	Trích xuất header: 6,738,415,616 params
Conflict	Card khai 70B nhưng file chỉ có 6.74B	Quyền ưu tiên cấu trúc (Measured) , lưu audit log

Ví Dụ: Xử Lý Conflict Giữa Declared Metadata & Đo Đặc Thực Tế

⚠ 1. Sai Lệch Khai Báo (Model Card)

- Repo fork đổi tên model thành `super-llama-70b`.
- Model Card khai báo: `parameters: 70B`.
- File nhị phân thực tế chỉ có 6.74B params.

🛡 2. Logic Resolution Của Scanner

- Parse header tensor → xác định **6.74B params**.
- **Measured > Declared** → kết luận 6.74B vào BOM.
- Ghi log: `rejected_claim: 70B (hf_card)`.

3 Trục Đánh Giá Dữ Liệu Cốt Lõi

? 1. EvidenceClass

Mức độ tin cậy (4 cấp):

- heuristic (0)
- declared (1)
- verified (2)
- measured (3)

→ Độ xác thực của claim.

📍 2. SourceRef

Nguồn dữ liệu (6 nguồn):

- local_binary
- hf_api
- hf_card
- local_config
- derived
- heuristic

🔗 3. Resolution

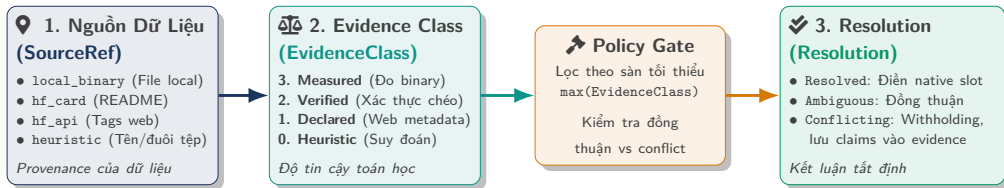
Trạng thái (4 loại):

- Resolved (Rõ ràng)
- Ambiguous (Đồng thuận)
- Conflicting (Tranh chấp)
- Unknown (Chưa rõ)

📄 Giá Trị Tính Toán Dẫn Xuất (Derived Fields)

$\text{VRAM} = \text{params} \times 2 \text{ bytes (BF16)}$ → tính trực tiếp từ giá trị đo đạc *measured* → giữ nguyên cấp tin cậy **Measured**.

Sơ Đồ Trực Quan: Luồng Phối Hợp 3 Trục Khái Niệm Bằng Chứng



💡 Tính Độc Lập Trục Giao (Orthogonality)

SourceRef biểu thị provenance vật lý; **EvidenceClass** xác định trọng số độ tin cậy; **Resolution** là kết quả logic tất định. Tuyệt đối không đồng nhất provenance với kết luận resolution.

Ví Dụ: Tại Sao Cùng 1 Request API Lại Tách 2 Nguồn Dữ Liệu?

```
{
  "id": "user/my-model",
  // 1. NGUON hf_api: Tac gia chon tren Web Settings
  "tags": ["license:apache-2.0", "text-generation"],

  // 2. NGUON hf_card: Boc tu ruot file README.md
  "cardData": {
    "license": "mit",
    "datasets": ["c4"]
  }
}
```

⚡ Nguy Cơ Mất Dữ Liệu Nếu Gộp Nguồn

- Hugging Face **không** tự động đồng bộ 2 trường này.
- Nếu gộp chung thành 1 nguồn hf: Giá trị apache-2.0 và mit sẽ đè mất nhau.
- **Giải pháp:** Tách riêng 2 nguồn hf_api (Tags) và hf_card (README frontmatter) → Engine tự động phát hiện mâu thuẫn nội bộ.

Phân Loại File & Giới Hạn Mức Tin Cậy (FileRole)

Vai Trò File	Mức Tin Cậy Max	Tạo Candidate?	Nguyên Tắc Xử Lý
Weight	measured	Có (Đầy đủ)	File chứa model weights → match hash xác định danh tính model.
Auxiliary	heuristic	Có (Bị chặn)	File cấu hình dùng chung → khóa trần heuristic tránh gán nhầm.
Media	N/A	Không	File ảnh, icon → 0 dùng để nhận diện model.
Archive	N/A	Không	File nén .zip, .tar → 0 dùng để nhận diện model.

Ví Dụ Trực Quan: Tại Sao Bắt Buộc Phân Loại FileRole?

1. File Weight (`model.safetensors`)

- Model weights 16 GB chứa cấu trúc tensor.
- Match SHA-256 → Xác định đúng model Meta-Llama-3-8B (Cấp Measured).

3. File Media (`logo.png` 300 byte)

- Xuất hiện ở **352 repos** khác nhau.
- Từ chối tạo candidate model → chỉ xuất component type: "file".

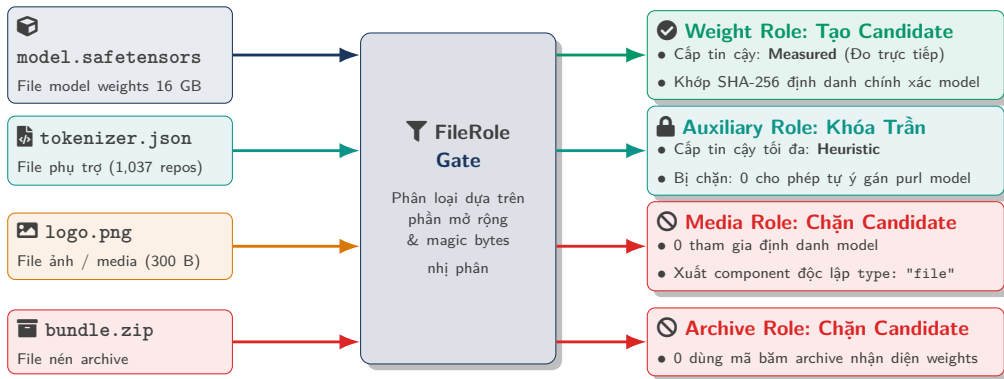
2. File Phụ Trợ (`tokenizer.json`)

- Dùng chung ở **1,037 repos** với 12 license.
- Khóa trần ở mức heuristic → không cho phép tự ý gán purl model cấp cao.

4. File Archive (`weights.zip`)

- File nén nhiều định dạng bên trong.
- Không dùng mã hash archive để nhận diện danh tính model.

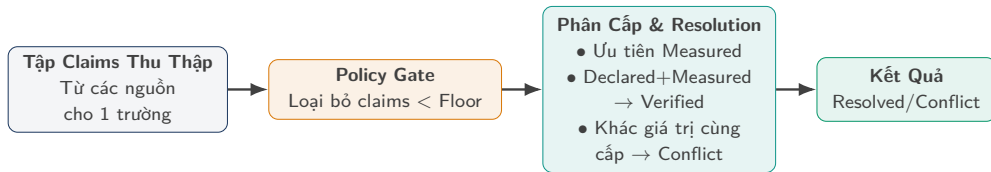
Sơ Đồ Trực Quan: Cổng Lọc Phân Cấp FileRole (File Gate)



💡 Nguyên Tắc Cô Lập Định Danh

Chỉ có file mang vai trò **Weight** mới có quyền định danh mô hình học máy. Các file phụ trợ, hình ảnh và tệp nén bị cô lập hoàn toàn để triệt tiêu nguy cơ gán nhầm metadata trên diện rộng.

Quy Trình Resolution Nội Bộ Repo (from_claims)



Loại Bỏ Claims Dưới Sàn Bằng Policy Gate

Đặt sàn = *declared* → loại bỏ phỏng đoán *heuristic* trước khi so sánh → ngăn chặn conflict giả tạo từ *heuristic*.

6 Tình Huống Xử Lý Thực Tế Của Thuật Toán from_claims

Tình Huống	Tập Claims Đầu Vào	Floor	Kết Quả Resolution
A. Đồng thuận	MIT (Card) + MIT (API)	heuristic	Resolved { MIT, declared
B. Xác thực chéo	4096 (Card) + 4096 (Header đo)	heuristic	Resolved { 4096, verified
C. Quyền ưu tiên cấu trúc	FP32 (Card) + BF16 (Header đo)	heuristic	Resolved { BF16, measured
D. Conflict nội bộ	MIT (Card) + Apache-2.0 (API)	heuristic	Conflicting (Withholding)
E. Policy Gate lọc Heuristic	MIT (Card) + GPL (Tên phỏng đoán)	declared	Resolved { MIT, declared
F. Không có dữ liệu	Trống 0 có claim nào	declared	Unknown

Cách Xử Lý Khi 1 File Trùng Ở Nhiều Repo (across_candidates)

1. Đánh Giá Tranh Chấp Trên Toàn Bộ Repo

- Duyệt và kiểm tra conflict trên toàn bộ N repos trước khi cắt giảm danh sách.
- 100% repo cùng license → Ambiguous(agreed) (điền native slot).
- Có repo lệch license → Conflicting (withholding native slot).

2. Thuật Toán Round-Robin Giữ Đủ License

- Thuật toán Round-Robin: Giới hạn 50 repo mẫu đại diện theo từng nhóm license.
- Lấy luân phiên đại diện từng nhóm.
- Đảm bảo giữ đủ 12/12 loại license xuất hiện.

⚡ Thách Thức: Lệch Tần Suất Dữ Liệu

- File tokenizer.json xuất hiện ở **1,037 repos** với 12 loại license.
- Các repo phổ biến nhất chỉ tập trung vào MIT và Apache-2.0.
- Nếu chỉ lấy mẫu theo số lượng → bỏ sót 10 loại license còn lại (GPL, BSD, CC-BY...)?

✓ Giải Pháp: Phân Bỏ Đại Diện Round-Robin

- Gom 1,037 repos vào 12 nhóm theo từng loại license.
- Nhật luân phiên: 1 MIT, 1 Apache, 1 GPL, 1 BSD...
- Lặp lại cho đến khi đủ 50 repo đại diện.
- → Đảm bảo **100% độ bao phủ cho 12/12 loại license!**

Sơ Đồ Trực Quan: Cơ Chế Phân Bỏ Đại Diện Round-Robin



💡 Nguyên Tắc Đại Diện Đa Dạng Hóa

Khi một tệp xuất hiện ở hàng nghìn kho lưu trữ, thuật toán Round-Robin ngăn chặn hiện tượng các kho lưu trữ phổ biến áp đảo danh sách, bảo đảm mọi loại giấy phép đều có đại diện để kiểm toán viên xem xét.

12 Tình Huống Thực Tế Khi Quét Model & JSON Mẫu

Tổng Hợp 12 Case Thực Tế Khi Quét Model

Ca	Tình Huống Thực Tế	Cấu Trúc File	Trạng Thái	Hành Vi JSON Đầu Ra
1	Khớp Đơn Nhất 1-1	1 File ↔ 1 Repo	Resolved	Điền đủ native slots, confidence: 1.0
2	Nhiều Repo Đồng Thuận	1 File → 3 Repos (cùng license)	Ambiguous(agreed)	Điền native slot licenses, candidateCount: 3
3	Conflict Giấy Phép Đa Repo	1 File 43 GiB → 3 Repos (khác license)	Conflicting	Withholding native slot, ghi nhận claims
4	Auxiliary File Trùng Lặp	tokenizer.json → 1,037 Repos	Conflicting	Round-Robin (50 candidate cover 12 licenses)
5	File Media Không Phải Model	logo.png → 352 Repos	FileRole::Media	Chặn tạo candidate model, 0 xuất purl
6	Thư Mục Chứa Sharded Weights	3 Shards + config.json	Resolved(verified)	Gom sharded weights, thăng cấp Verified
7	Thư Mục Multi-Group (Quant)	7 GGUFs + 1 config.json	Multi-Group	Root sang type: "file", withholding shapes
8	File Chưa Từng Được Crawl	Hash Miss (File nội bộ)	Unknown	Binary sniffer đo parameters cấp measured
9	Conflict Nội Bộ Repo	hf_api (Apache) vs hf_card (MIT)	Conflicting	Withholding native slot, lưu claims vào evidence
10	Conflict Declared vs Measured	Card khai 70B vs Header đo 6.74B	Resolved(measured)	Quyền ưu tiên cấu trúc (Measured), log audit
11	File Weights 0-Byte Không Hợp Lệ	File .bin 0 byte	EMPTY_SHA256	Bỏ qua mã băm rỗng, cảnh báo tệp không hợp lệ
12	Model ONNX Tách Rời Trọng Số	model.onnx + _data_N	Resolved(sharded)	Gom nhóm các file phân tách vào 1 model group

Ca 1: Khớp Đơn Nhất 1–1 (Single Model Match)

✓ Đặc Tính & Cách Xử Lý

- File `model.safetensors` khớp đúng 1 repo.
- Đầy đủ bằng chứng xác thực → điền toàn bộ native slots.
- Khớp SHA-256 tuyệt đối với cơ sở dữ liệu → gán độ tin cậy `confidence: 1.0`.

```
{
  "bomFormat": "CycloneDX", "specVersion": "1.6",
  "serialNumber": "urn:uuid:3e6717a6-...",
  "components": [{
    "bom-ref": "model-scan-001",
    "type": "machine-learning-model",
    "name": "Meta-Llama-3-8B", "group": "meta-llama",
    "purl": "pkg:huggingface/meta-llama/Meta-Llama-3-8B@e9149a1",
    "licenses": [{"license": {"name": "llama3"}}],
    "evidence": {
      "identity": {
        "field": "purl", "confidence": 1.0,
        "methods": [{"technique": "hash-comparison", "confidence": 1.0}]
      }
    }
  ]
}
```

Ca 2: Nhiều Repo Nhưng Cùng Khai Báo (Multi-Repo Unanimous)

Đặc Tính & Cách Xử Lý

- Model weights xuất hiện ở 3 mirror repos.
- Cùng khai license: llama3 và tác giả meta-llama.
- Ambiguous(agreed) → vẫn điền native slot licenses.

```
{  
  "components": [{  
    "bom-ref": "model-scan-002",  
    "type": "machine-learning-model",  
    "name": "Llama-3-8B",  
    "licenses": [{"license": {"name": "llama3"}}],  
    "evidence": {  
      "identity": {  
        "field": "purl",  
        "candidateCount": 3,  
        "resolution": "ambiguous",  
        "agreement": true,  
        "methods": [{"technique": "hash-comparison", "confidence": 1.0}]  
      }  
    }  
  ]  
}
```


Ca 3: Conflict Giấy Phép Đa Repo (1-N Candidates)

⚠ Đặc Tính & Cách Xử Lý

- File 43 GiB nằm ở 3 repos (Repo A: *other*, Repo B: *MIT*, Repo C: *trống*).
- Withholding native slot (licenses: []), không suy đoán chủ quan.
- Lưu đầy đủ các claims độc lập vào evidence.licenses[].

```
{
  "components": [{
    "type": "machine-learning-model", "name": "SomeModel",
    "licenses": [], // De trong do bat dong thong tin
    "evidence": {
      "identity": {
        "resolution": "conflicting", "conflicts": ["license"]
      },
      "licenses": [
        {"license": {"name": "other"}, "source": "repo-a", "downloads": 5200000},
        {"license": {"id": "MIT"}, "source": "repo-b", "downloads": 200000}
      ]
    }
  }]
}
```

Ca 4: File Phụ Trùng Hàng Nghìn Repo (tokenizer.json)

Đặc Tính & Cách Xử Lý

- File tokenizer.json xuất hiện ở 1,037 repos.
- Khóa trồn ở mức *heuristic* → 0 gán nhầm danh tính.
- Thuật toán Round-Robin lấy 50 đại diện giữ đủ 12 loại license.

```
{
  "components": [{
    "name": "tokenizer", "licenses": [],
    "evidence": {
      "identity": {
        "candidateCount": 1037,
        "resolution": "conflicting", "conflicts": ["license"]
      },
      "licenses": [
        // 50 đại diện Round-Robin cân bằng từ 12 nhóm license
        {"license": {"id": "Apache-2.0"}, "source": "repo-1"},
        {"license": {"id": "MIT"}, "source": "repo-2"},
        {"license": {"id": "GPL-3.0"}, "source": "repo-3"}
        // Bao phủ đầy đủ 12 nhóm license
      ]
    }
  ]
}
```

⊘ Đặc Tính & Cách Xử Lý

- File logo.png 300 byte trùng ở 352 repos.
- Nhận diện là Media → 0 tạo candidate model.
- Output ra type: "file", 0 gán purl model.

```
{  
  "components": [{  
    "bom-ref": "file-scan-005",  
    "type": "file",  
    "name": "huggingface_logo.png",  
    "hashes": [{  
      "alg": "SHA-256",  
      "content": "def456..."  
    }]  
    // Không gán purl và modelCard  
    // Tu chời suy diên sai danh tính model  
  }]  
}
```

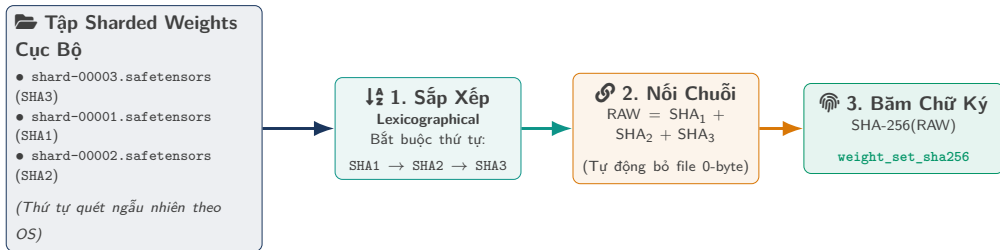
Ca 6: Thư Mục Chứa Sharded Weights

Đặc Tính & Cách Xử Lý

- Thư mục chứa 3 sharded weights .safetensors + config.json.
- Gom sharded weights → tính weight_set_sha256.
- Đối chiếu với config.json → thăng cấp lên **Verified**.

```
{
  "metadata": {
    "component": {
      "type": "machine-learning-model",
      "name": "Meta-Llama-3-8B",
      "purl": "pkg:huggingface/meta-llama/Meta-Llama-3-8B@e9149a1"
    }
  },
  "components": [{
    "modelCard": {
      "modelParameters": {
        "architecture": "LlamaForCausalLM",
        "parameters": 8030000000 // Dat cap Verified qua cross-verify
      }
    }
  ]
}
```

Ví Dụ Trực Quan: Cách Gom Nhóm & Tính Chữ Ký Sharded Weights



✓ Tính Tất Định Tuyệt Đối (Determinism)

Cho dù hệ điều hành (Windows NTFS, Linux ext4, macOS APFS) liệt kê file theo thứ tự nào, bước **Lexicographical sort** đảm bảo chuỗi băm RAW luôn giống nhau → Chữ ký `weight_set_sha256` trùng khớp byte-for-byte trên mọi hệ thống.

Ca 7: Thư Mục Chứa Nhiều Bản Quantization (GGUF Multi-Group)

📁 Đặc Tính & Cách Xử Lý

- Thư mục chứa 7 file GGUF quantization + 1 config.json.
- Thư mục Multi-Group → Root chuyển thành type: "file".
- Withholding các trường hình thái model vào withheld.

```
{
  "metadata": {
    "component": {
      "type": "file", // Chuyển từ machine-learning-model
      "name": "my_models_dir"
    }
  },
  "properties": [
    {"name": "aibom:withheld", "value": "architecture,param_count,context_length"}
  ]
  // 7 cum model con duoc mo ta doc lap trong envelope report/2
}
```

Ví Dụ Trực Quan: Xử Lý Thư Mục 7 Bản Quantization GGUF (Multi-Group)

Thư Mục my_models/

- └ llama-3-q2_k.gguf (4 GB, Q2)
- └ llama-3-q4_k_m.gguf (6 GB, Q4)
- └ llama-3-q8_0.gguf (10 GB, Q8)
- └ ... (tổng cộng 7 bản quant)
- └ config.json (**Ghi shape FP16!**)

⊗ Nguy Cơ Áp Nhầm Config

- Nạp config.json vào root
- Gán thông số FP16 (32 GB VRAM) cho cả file Q2_K (chỉ cần 4 GB VRAM)
- → Sai lệch thông số phần cứng!

🛡️ Giải Pháp Cách Ly AI-BOM

- Root chuyển thành type: "file"
 - Withholding shape chung vào withheld
 - Tách 7 bản GGUF thành 7 components con
- với thông số đo đạc độc lập!

🛡️ Nguyên Tắc An Toàn Metadata

Khi một thư mục chứa nhiều model có cấu hình khác nhau, tuyệt đối không cho phép file cấu hình ngoài (như config.json) áp đặt thuộc tính chung lên toàn bộ thư mục.

🔍 Đặc Tính & Cách Xử Lý

- File binary nội bộ chưa có trên Hugging Face.
- Danh tính → resolution: unknown.
- Binary sniffer parse header file để đo trực tiếp thông số ở cấp **Measured**.

```
{
  "components": [{
    "type": "machine-learning-model",
    "name": "unknown-model",
    "modelCard": {
      "modelParameters": {
        "parameters": 6738415616, // Do trực tiếp từ header (Measured)
        "dataType": "BF16" // Do trực tiếp từ header (Measured)
      }
    },
    "evidence": {
      "identity": {"resolution": "unknown"}
    }
  }]
}
```


Ca 9: Conflict Nội Bộ Repo (API Tags vs README)

⚡ Đặc Tính & Cách Xử Lý

- Cùng 1 repo: API khai Apache-2.0 nhưng Card khai MIT.
- Tách riêng từng nguồn dữ liệu → phát hiện conflict.
- Withholding native slot, lưu cả 2 nguồn claims vào evidence.

```
{
  "components": [{
    "name": "my-model", "licenses": [],
    "evidence": {
      "identity": {"resolution": "conflicting", "conflicts": ["license"]},
      "licenses": [
        {"license": {"id": "Apache-2.0"}, "source": "hf_api", "class": "declared"},
        {"license": {"id": "MIT"}, "source": "hf_card", "class": "declared"}
      ]
    }
  ]
}
```

Ca 10: Conflict Giữa Declared Metadata & Đo Đặc Cấu Trúc

⚠️ Đặc Tính & Cách Xử Lý

- Model Card khai nhầm:
param_count: 70B.
- Binary sniffer parse tensor header
đếm ra: **6.74B** params.
- **Structural Override (Measured > Declared)**: Chốt 6.74B, ghi log
audit.

```
{
  "components": [{
    "modelCard": {
      "modelParameters": {
        "parameters": 6738415616 // Chốt theo kết quả đo lường
      }
    },
    "properties": [
      // Log khai 70B bị loại và lưu vào audit log
      {"name": "aibom:audit.rejected_claim", "value": "param_count:70B (hf_card)"}
    ]
  }]
}
```

! Đặc Tính & Cách Xử Lý

- Thư mục chứa file weights 0 byte.
- Loại bỏ hash rỗng khỏi nhận diện model.
- Cảnh báo "No valid weight files found", không gán candidate.

```
{
  "metadata": {
    "component": {"type": "file", "name": "empty_model_dir"}
  },
  "properties": [
    {
      "name": "aibom:warning",
      "value": "No valid weight files found. Directory does not identify a model."
    }
  ]
}
```

Đặc Tính & Cách Xử Lý

- Model ONNX chia `model.onnx` + `model.onnx_data_N`.
- Tự động cắt hậu tố số $\rightarrow$ gom 245 file split (332 GB) vào 1 weight group.

```
{
  "components": [{
    "type": "machine-learning-model",
    "name": "gemma-4-onnx",
    "properties": [
      {"name": "aibom:weight_set_sha256", "value": "a1b2c3..."},
      {"name": "aibom:group_members", "value":
        "model.onnx,model.onnx_data_1,model.onnx_data_2"}
    ]
  }]
}
```

Cấu Trúc JSON Đầu Ra: Chuẩn Quốc Tế & Nội Bộ

8 Quy Tắc Quyết Định Dữ Liệu Xuất Ra JSON

#	Tên Quy Tắc	Hành Vi Xuất Dữ Liệu Thực Tế
1	Native Slot Gating	Conflict → withholding native slot, lưu claims vào evidence.
2	Structural Override	Đo đạc measured ưu tiên tuyệt đối trước declared.
3	FileRole Ceiling	Auxiliary khóa trần ở heuristic; Media chặn tạo candidate.
4	Round-Robin Coverage	Giới hạn 50 candidate đại diện nhưng bảo toàn 12 nhóm license.
5	Full Candidate Check	Kiểm tra conflict trên toàn bộ candidate trước khi cắt giảm.
6	Policy Gate	Lọc claims < floor trước khi resolution để chống conflict giả.
7	Multi-Group Withholding	Thư mục Multi-Group: Root là file, đưa thông số model vào withheld.
8	SPDX Auto-Routing	Khớp SPDX → license.id; Không khớp → license.name.

1. Chuẩn Quốc Tế CycloneDX 1.6

- Chuẩn mở ML-BOM của OWASP.
- Tương thích ngay với các tool kiểm toán và SIEM quốc tế.
- Định danh components, model parameters, datasets.

2. Định Dạng Đầy Đủ report/2

- Định dạng mở rộng nội bộ phục vụ audit sâu.
- Phân rõ chi tiết từng nhóm weights (groups []).
- Lưu near_misses[], withheld[] và completeness score.

Ví Dụ: Cấu Trúc Báo Cáo Nội Bộ Envelope (report/2)



Đặc Điểm Tầng Envelope

- **Hai tầng phân cấp:** Tầng ngoài bọc báo cáo audit nội bộ (report/2), tầng trong chứa payload CycloneDX 1.6.
- **Theo dõi cụm weights:** Mảng groups[] theo dõi từng nhóm weights và chữ ký weight_set_sha256.
- **Minh bạch nguồn gốc:** Trường provenance ghi rõ từng thuộc tính lấy từ đâu.
- **Chấm điểm dữ liệu:** Trường completeness_score đánh giá 40 tiêu chí chất lượng.

```
{
  "schema": "opswat.aibom.report/2",
  "engine_version": "0.1.0",
  "scan_target": "/data/models/Meta-Llama-3-8B",
  "scan_timestamp": "2026-09-01T12:00:00Z",
  "deterministic": true,
  "completeness_score": 0.92,
  "groups": [{
    "group_name": "Meta-Llama-3-8B-Weights",
    "weight_set_sha256": "4a7d8b2e1f9c3a0b...",
    "member_count": 4,
    "total_bytes": 16060522496,
    "resolution": "resolved",
    "candidates": [
      "pkg:huggingface/meta-llama/Meta-Llama-3-8B@e9149a1"
    ],
    "provenance": {
      "license": "hf_card",
      "param_count": "local_binary",
      "architecture": "local_config"
    }
  ]},
  "cyclonedx": { /* CycloneDX 1.6 ML-BOM Payload */ }
}
```


Ảnh Xạ Dữ Liệu Vào Chuẩn Quốc Tế CycloneDX 1.6

Trường CycloneDX 1.6 Chuẩn	Nguồn Dữ Liệu & Ảnh Xạ Hệ Thống
hashes[]	SHA-256 từng file và chữ ký weight_set_sha256
name / group	Tên model / Namespace tác giả (<i>meta-llama</i>)
version / purl	Commit SHA / Purl chuẩn pkg:huggingface/<author>/<name>@<sha>
licenses[]	License kết luận (SPDX ID hoặc Custom Name)
modelCard.modelParameters	architectureFamily, modelArchitecture, task, parameters, datasets[]
evidence.identity	Method định danh (<i>hash-comparison</i> / <i>binary-analysis</i>), confidence: 1.0
evidence.licenses[]	Danh sách claims khi có tranh chấp license
Thuộc Tính Mở Rộng ('aibom:*' / 'lms:*')	Ý Nghĩa Kỹ Thuật Chuyên Biệt
aibom:quantization	Kiểu lượng tử hóa (<i>Q4_K_M</i> , <i>Q8_0</i> , <i>FP16</i>)
lms:param_count / lms:tensor_count	Tổng params và tổng tensors đếm trực tiếp từ header file
lms:context_length / lms:layer_count	Context length tối đa và số layer kiến trúc
aibom:withheld	Danh sách trường bị ẩn để tránh sai lệch trong thư mục đa model

Ví Dụ: Payload Chuẩn Quốc Tế CycloneDX 1.6 ML-BOM

✓ Chuẩn Hóa CycloneDX 1.6

- **UUIDv8 tất định:** serialNumber theo RFC 9562, khóa timestamp → byte-for-byte.
- **Không trùng lặp:** metadata.component không bom-ref tránh xung đột unique.
- **Khẳng định danh tính:** confidence: 1.0 tuyệt đối với băm SHA-256.
- **14 Extensions:** Đo đặc thực tế (lms:*) và lượng tử hóa (aibom:*)

```
{
  "bomFormat": "CycloneDX", "specVersion": "1.6",
  "serialNumber": "urn:uuid:3e6717a6-2009-8786-9051-7f888eb1a57e",
  "components": [{
    "bom-ref": "model-artifact-root", "type": "machine-learning-model",
    "name": "Meta-Llama-3-8B",
    "purl": "pkg:huggingface/meta-llama/Meta-Llama-3-8B@e9149a1",
    "licenses": [{ "license": { "name": "llama3" } }],
    "modelCard": {
      "modelParameters": {
        "architectureFamily": "llama",
        "parameters": 8030261248 // Measured
      }
    },
    "evidence": {
      "identity": {
        "field": "purl", "confidence": 1.0,
        "methods": [{ "technique": "hash-comparison", "confidence": 1.0 } ]
      }
    },
    "properties": [
      { "name": "aibom:quantization", "value": "BF16" },
      { "name": "lms:param_count", "value": "8030261248" }
    ]
  }]
}
```

⚠ Yêu Cầu Của Schema CycloneDX 1.6

Validator sẽ báo lỗi nếu `license.id` chứa giá trị nằm ngoài từ điển SPDX.

✓ Tự Động Điều Hướng 811 SPDX IDs

- Khớp từ điển SPDX → ghi vào `license.id`.
- Tên riêng tự đặt (llama3) → ghi vào `license.name`.
- 100% pass CycloneDX schema validator.

🌀 Sinh ID Tất Định Theo RFC 9562 UUIDv8

- **UUIDv4 vs UUIDv8:** UUIDv4 là ngẫu nhiên; UUIDv8 cho phép nhúng payload tùy biến (SHA-256 + timestamp cố định).
- `serialNumber` sinh tất định từ 16 byte đầu SHA-256 của file.
- Khóa timestamp → quét 2 lần cùng 1 target cho output JSON trùng byte-for-byte.

Đánh Giá Độ Đầy Đủ Của Metadata (Completeness Score)

📝 40 Trường Đánh Giá

Kiểm tra 4 nhóm thông tin chính:

- **Danh tính:** name, author, purl, sha256
- **Pháp lý:** license, license_type
- **Thông số kỹ thuật:** param_count, dtype, architecture, context_length
- **Nguồn gốc:** base_model, datasets

% Công Thức Tính Completeness Score

$$\text{completeness_score} = \frac{\sum (\text{field có data} \times \text{weight})}{\sum \text{tổng weight}}$$

- Đo lường độ đầy đủ thông tin của model trong BOM.
- Tự động cảnh báo khi model thiếu license hoặc thông số quan trọng.

Đọc Header Binary, Chống Mã Độc & Kết Quả Test

Kiến Trúc Quét 2 Giai Đoạn (Two-Phase Scan Pipeline)

Phase 1: Fast Structural Sniffing (< 5ms)

Mục tiêu: Triage nhanh & Trích xuất hình thái

- Đọc vài KB đầu file (header length + JSON metadata).
- Trích xuất cấu trúc tensor: params, dtype, context length (Cấp Measured).
- Duyệt opcode tĩnh file Pickle → loại trừ mã độc RCE.
- Tiêu tốn < 50MB RAM, thực thi trong < 5ms.

Phase 2: Streaming Identity Hashing

Mục tiêu: Tính SHA-256 tra cứu Cache L2

- Đọc tuần tự theo buffer I/O chunking (64KB / 1MB).
- Tính SHA-256 từng file & weight_set_sha256 mà 0 nạp toàn bộ file vào RAM.
- Tra cứu exact hash vào Layer 2 SQLite (< 1ms).
- Quét file 40 GB an toàn, 0 tràn bộ nhớ (OOM).

Đọc Header File Binary Cực Nhanh (< 5ms, Không Load Weights)

Định Dạng	Module Rust	Cách Sniff Header Tốc Độ Cao (< 5ms)
GGUF	<code>formats/gguf.rs</code>	Đọc magic bytes GGUF, parse các cặp KV (<i>architecture</i> , <i>context_length</i> , quantization <i>Q4_K_M</i>).
Safetensors	<code>formats/safetensors.rs</code>	Đọc 8 bytes đầu lấy độ dài header, parse JSON metadata và danh sách tensor shapes/dtypes.
ONNX	<code>formats/onnx.rs</code>	Parse protobuf header: graph nodes, initializer tensor shapes.
HDF5	<code>formats/hdf5.rs</code>	Duyệt cây B-Tree / Group header trích xuất Keras weights.

Ví Dụ Trực Quan: Đọc Header File 40 GB Trong Dưới 5ms

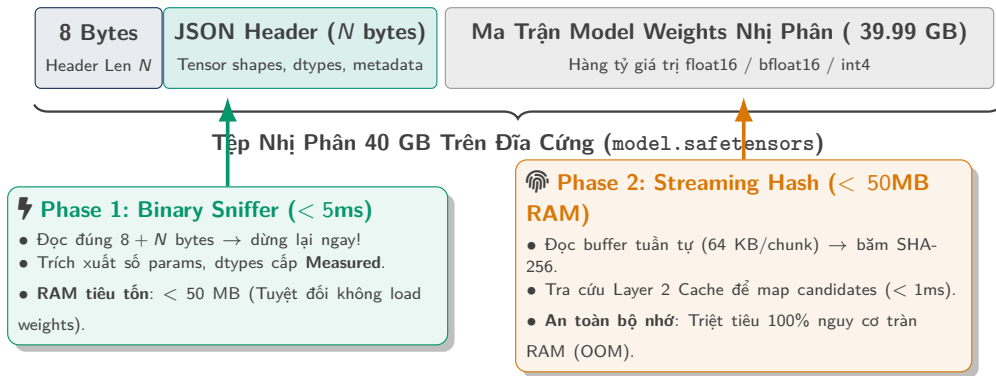
⊗ Phương Pháp Eager Loading (Nạp Toàn Bộ)

- Dùng Python/PyTorch load toàn bộ file weights 40 GB vào RAM.
- Thời gian thực thi: **30–60 giây**.
- Tiêu tốn > 40 GB RAM, dễ crash/OOM khi scan thư mục lớn.

⚡ Cơ Chế Binary Sniffer (Zero-Copy Header Parsing)

- Đọc đúng **8 bytes** đầu để lấy độ dài header (N bytes).
- Đọc tiếp N bytes JSON metadata của header.
- Thời gian thực thi: < 5 ms.
- Tiêu tốn < 50 MB RAM, trích xuất đủ tensor shapes và dtypes.

Sơ Đồ Trực Quan: Cấu Trúc File Nhị Phân & Quét 2 Giai Đoạn



💡 Giải Quyết Hoàn Toàn Nghịch Lý Hiệu Năng

Tách biệt hoàn toàn giữa **trích xuất thuộc tính hình thái** (chỉ đọc header trong < 5ms) và **định danh cryptographic** (đọc luồng chunking để tính SHA-256 mà không nạp weights vào RAM).

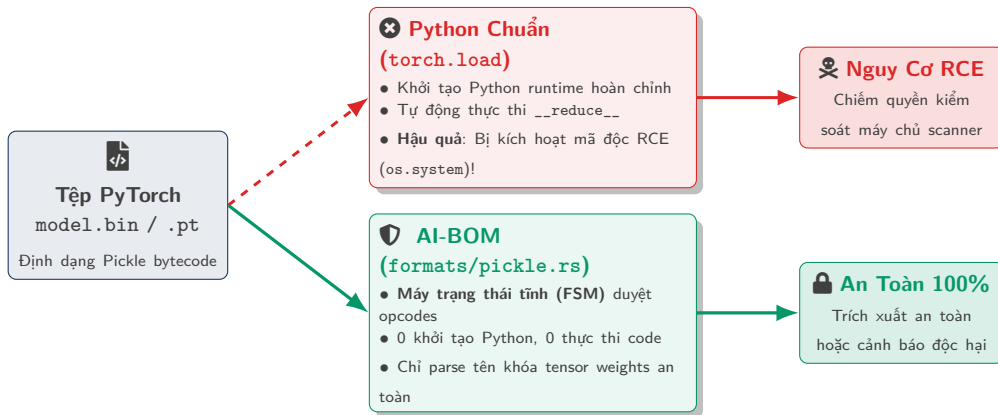
Rủi Ro Mã Độc Trong File Pickle

File PyTorch Pickle có thể chứa bytecode độc hại → rủi ro thực thi mã tùy ý (RCE) nếu nạp bằng Python interpreter chuẩn.

Bộ Phân Tích Opcode Tĩnh Trong `formats/pickle.rs`

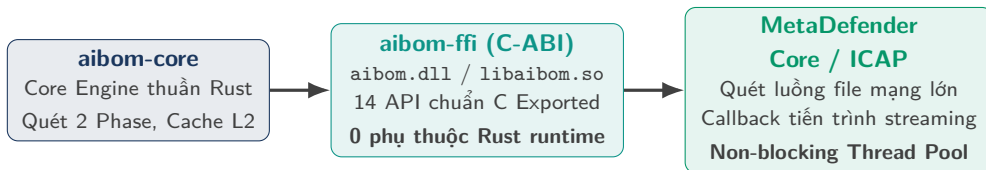
- Máy trạng thái duyệt từng opcode nhị phân tĩnh trong file pickle.
- 0 khởi tạo Python runtime, 0 nạp đối tượng thực thi.
- Trích xuất an toàn tensor keys → **Triệt tiêu nguy cơ thực thi mã độc (RCE).**

Sơ Đồ Trực Quan: Máy Trạng Thái Tĩnh Quét Mã Độc Pickle



✓ Bảo Mật Tuyệt Đối Chuỗi Cung Ứng AI

Bộ phân tích opcode tĩnh của AI-BOM giải mã các chỉ lệnh nhị phân cấp thấp mà không bao giờ kích hoạt bất kỳ lệnh gọi hệ thống nào, ngăn chặn triệt để các tệp weights độc hại tấn công hạ tầng doanh nghiệp.



🔧 Đặc Điểm Nhúng Vào Scanning Engine

- **C-ABI tương thích trực tiếp**: Nhúng dạng shared library vào C++, C#, Go, Python của MetaDefender.
- **Xử lý file lớn qua mạng (ICAP)**: Hỗ trợ streaming hash chunk-by-chunk với progress callback → không phong tỏa thread pool chính khi tải file model 50 GB.

Kết Quả Test Suite (366 Tests) & Benchmark Hiệu Năng

☰ Kết Quả Test Suite

Chạy tự động qua `acceptance.ps1`:

1. **Workspace Tests**: 366 tests pass 100% (0 fail, 0 skip).
2. **Corpus Assertions**: 49,992 models, DDL 40 cột.
3. **Determinism**: So sánh byte-for-byte.
4. **CycloneDX 1.6**: JSON Schema Pass 100%.
5. **Regressions**: 8 tests chống lỗi hồi quy cho các trường hợp đặc biệt.
6. **Clippy Hygiene**: 0 warning (-D warnings).

🕒 Benchmark Hiệu Năng

- **Query hash SQLite**: < 1 ms.
- **Sniff header (file 40 GB)**: < 5 ms.
- **RAM runtime**: < 50 MB.
- **Offline Rebuild**: Rebuild 50k models trong vài phút với 0 request mạng.

4 Thách Thức Kỹ Thuật Đã Được Xử Lý Triệt Để

#	Thách Thức Kỹ Thuật	Giải Pháp Đã Xử Lý
1	Sốt file ONNX bị chia nhỏ	Regex cắt hậu tố <code><digits></code> → gom 245 file split (332 GB) vào đúng model group.
2	Xử lý file weights 0-byte không hợp lệ	Loại bỏ mã hash <code>EMPTY_SHA256</code> khỏi danh tính weight set.
3	Tránh ô nhiễm metadata thư mục Multi-Group	Gỡ shape fields khỏi root component, đưa vào danh sách <code>withheld</code> .
4	4,950 model khai license "other"	Tách cột <code>license_name</code> & <code>license_link</code> → chuẩn hóa 3,466 models sang tên license chuẩn.

🔗 Giá Trị Cốt Lõi Của AI-BOM 0.1.0

1. **Hệ thống 4 cấp Evidence Class:** Đo đặc cấu trúc binary trực tiếp > Declared metadata trên web, bảo đảm tính xác thực khoa học.
2. **SQLite 2 tầng:** Quét 100% offline không phụ thuộc mạng, nén 50k models còn 18 KB/model (912 MB).
3. **Xử lý 12 tình huống thực tế:** 1-N fan-out, multi-group quantization, sharded weights, conflict license.
4. **Chuẩn hóa quốc tế:** Hỗ trợ chuẩn CycloneDX 1.6 ML-BOM và format mở rộng nội bộ.

🗺️ Roadmap Kế Tiếp

Hoàn thiện Type 1 Final → Batch 4 (Delta sync & Cache lifecycle) → Batch 5 (Dataset catalog) → Batch 6.1 (Verified score).



Thảo Luận & Q&A

AI-BOM Architecture & Schema Specification · Version 0.1.0